

Task 4: OOPs concepts – Data hiding

Now that your fundamental structural framework is locked down, let me know if you would like to proceed with **The Data Hiding Step**: Converting public String name to private String name and creating an official **Getter Method** (public String getName()) to implement proper professional Java security standards.

- **Private Fields:** name and grades fields are changed to private. The main method can no longer access activeStudent.name or activeStudent.grades.length directly without causing a compilation error.
- **Getter Methods:** We introduce getName() and getGrades() to allow controlled, read-only access to our private fields. By using **Getters**, your main program gets the variables it needs, but the object maintains absolute authority over its internal state.
- **Zero Code Discarded:** The fixed SUBJECTS array layout, error-trapping inputs, and formatting mechanics remain fully intact

```
package Student_dataHiding;

import java.util.Scanner;

public class Student_dataHiding {
    public static final String[] SUBJECTS = {"Math", "Science", "Social", "English", "French"};

    private String name;
    private int[] grades;

    public Student_dataHiding(String rawName, Scanner inputScanner) {
        this.name = formatName(rawName);
        this.grades = new int[SUBJECTS.length];
        fillGradesArray(inputScanner);
    }

    public String getName() {
        return this.name;
    }

    public int[] getGrades() {
        return this.grades;
    }

    public boolean qualifiesForHonorRoll() {
        return calculateAverage() >= 90.0;
    }
}
```

```
private String formatName(String rawName) {
    String cleanName = rawName.trim();
    if (cleanName.isEmpty()) return "";
    return cleanName.substring(0, 1).toUpperCase() + cleanName.substring(1);
}

private void fillGradesArray(Scanner inputScanner) {
    System.out.println("\n--- Entering Grades for Class Schedule ---");
    for (int i = 0; i < this.grades.length; i++) {
        System.out.print("Enter text grade for " + SUBJECTS[i] + ": ");
        String textGrade = inputScanner.next();
        try {
            this.grades[i] = Integer.parseInt(textGrade);
        } catch (NumberFormatException e) {
            System.out.println("Invalid text format detected! Setting " + SUBJECTS[i] + " to 0%.");
            this.grades[i] = 0;
        }
    }
}

public double calculateAverage() {
    int sum = 0;
    for (int score : this.grades) {
        sum += score;
    }
    return (double) sum / this.grades.length;
}
```

```

import java.util.Scanner;
import Student_dataHiding.Student_dataHiding;

// — MAIN RUNNER EXECUTION SCRIPT —
public class Task4_DataHiding {
    public static void main(String[] args) {
        Scanner inputScanner = new Scanner(System.in);

        // --- DATA COLLECTION INTAKE ---
        System.out.println("=== Student Data Entry System ===");
        System.out.print("Enter raw student name: ");
        String rawName = inputScanner.nextLine();

        // --- OBJECT BIRTH SYNTHESIS ---
        // We instantiate the object. It handles text cleaning and sequential
        // subject tracking completely on its own inside the constructor!
        Student_dataHiding activeStudent = new Student_dataHiding(rawName, inputScanner);

        // --- FINAL REPORT CARD PROFILE GENERATION ---
        System.out.println("\n=====");
        System.out.println("    ACADEMIC REPORT    ");
        System.out.println("=====");
        System.out.println("Student Profile: " + activeStudent.getName());
        System.out.println("Total Classes: " + activeStudent.getGrades().length);

        // Call instance method to compute arithmetic output
        double computedAverage = activeStudent.calculateAverage();
        System.out.println("Final Average: " + computedAverage + "%");
        System.out.println("=====");

        inputScanner.close();
    }
}

```

